

Définition, contrôle et manipulation

1. Administration PostgreSQL

1.1. Création, modification et suppression

Les commandes suivantes sont valables pour les bases, les schémas, les tables et les rôles.

Création		<code>CREATE TABLE my-table ();</code>
Renommage		<code>ALTER TABLE my-table-1 RENAME TO my-table-2;</code>
Changement de propriétaire	de	<code>ALTER TABLE my-table OWNER TO my-role;</code>
Attribution de droits	de	<code>GRANT ALL ON TABLE my-table TO my-role;</code>
Révocation de droits	de	<code>REVOKE ALL ON TABLE my-table FROM my-role;</code>
Suppression		<code>DROP TABLE my-table;</code>

1.2. Sélection d'un schéma

Voir les schémas actifs	<code>SHOW search-path;</code>
Se placer dans le schéma <code>my_schema</code>	<code>SET search-path TO my-schema;</code>

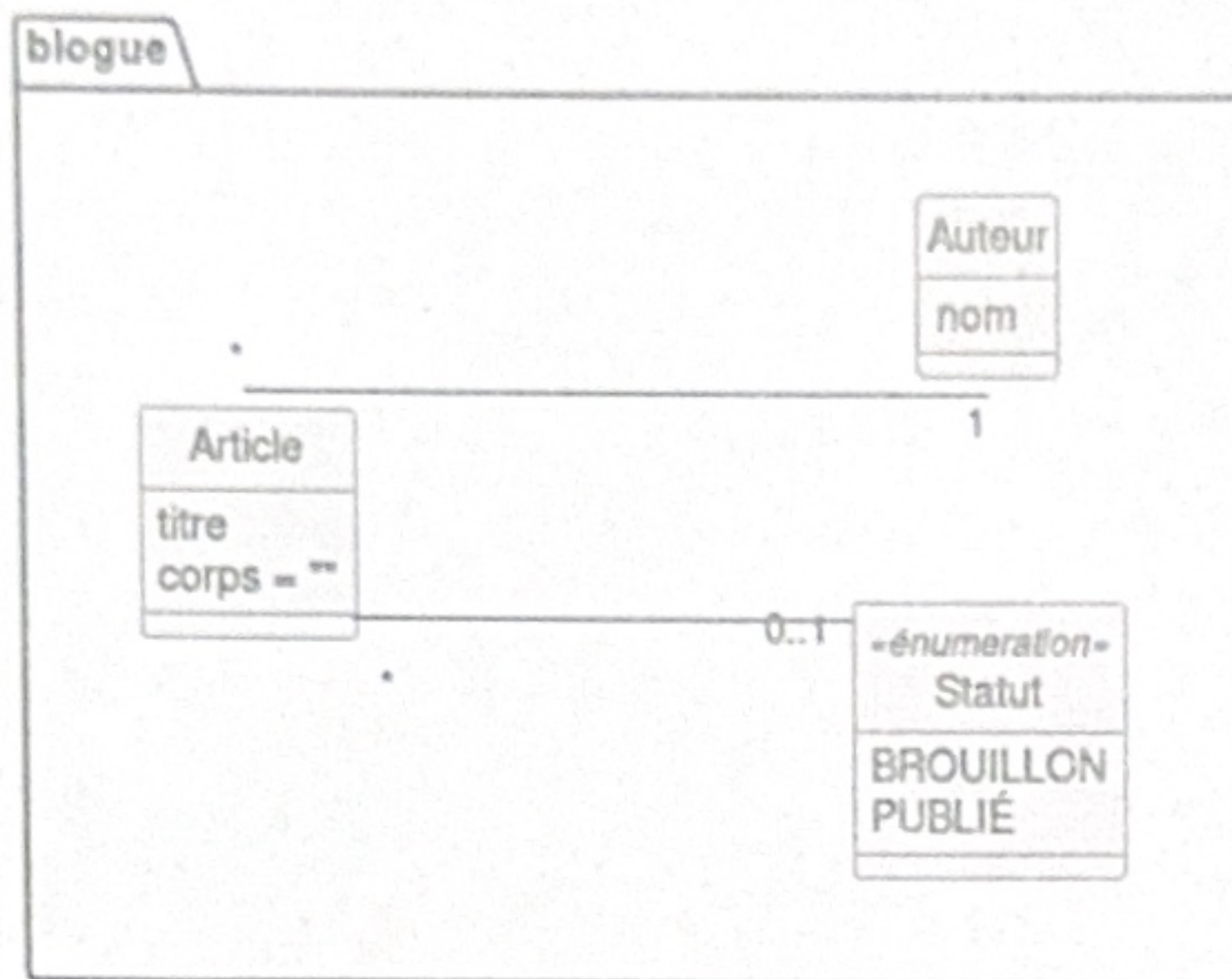
1.3. Remaniement d'une table

Ajouter un champ <code>my-field</code> de type <code>TEXT</code> à <code>my-table</code>	<code>ALTER TABLE my-table ADD COLUMN my-field TEXT;</code>
Supprimer le champ <code>my-field</code> de <code>my-table</code>	<code>ALTER TABLE my-table DROP COLUMN my-field;</code>
Renommer le champ <code>my_field</code> en <code>my_attribute</code>	<code>ALTER TABLE my-table RENAME my-field TO my-attribute;</code>
Ajouter une contrainte de non nullité	<code>ALTER TABLE my-table ALTER COLUMN my-column SET NOT NULL;</code>
Ajouter une valeur par défaut	<code>ALTER TABLE my-table ALTER COLUMN my-column SET DEFAULT '';</code>
Supprimer la contrainte <code>my_constraint</code> de <code>my-table</code>	<code>ALTER TABLE my-table DROP CONSTRAINT my-constraint</code>
Ajouter une contrainte de clé étrangère <code>table-2(k)</code> sur <code>table-1</code>	<code>ALTER TABLE table_1 ADD CONSTRAINT table_1_fkey FOREIGN KEY (k) REFERENCES table_2(k) ON UPDATE CASCADE;</code>

2. Types de base de PostgreSQL

INTEGER (INT)	nombres entiers	TEXT	chaînes de caractères
DECIMAL (DEC)	nombres décimaux	DATE	dates

3. Définition des tables



```

CREATE SCHEMA blog;

CREATE TYPE blog.STATUS AS ENUM('brouillon', 'publié');

CREATE TABLE blog.authors(
    author-id SERIAL PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE blog.articles(
    article-id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    body TEXT DEFAULT '',
    author INT NOT NULL REFERENCES blog.author(author-id)
    ON UPDATE CASCADE
    ON DELETE SET NULL,
    status blog.STATUS
);
  
```

4. Manipulation des données

Insérer deux brouillons vides dans la table <code>blog.articles</code> .	<pre> INSERT INTO blog.article(title, author, status) VALUES ('Tux porte disques', 1, 'brouillon'), ('Tux peut-être mort', 1, 'brouillon'); </pre>
Supprimer l'article n°5 de la table <code>blog.articles</code> .	<pre> DELETE FROM blog.articles WHERE article-id = 5; </pre>
Renommer l'article n°5 en «Tux retrouvé».	<pre> UPDATE blog.articles SET title = 'Tux retrouvé' WHERE article-id = 5; </pre>

Interrogation des données

competitors

competitor_id	name	country	score
1	Tux	LX	3200
2	Linus	LX	3200
3	Bill	US	150
4	Boule	FR	150
5	Steve	US	399

(5 lignes)

1. Requêtes simples

- Sélectionner les champs **name** et **country** de la table **competitors** :

name	country
Tux	LX
Linus	LX
Bill	US
Boule	FR
Steve	US

(5 lignes)

```
SELECT mt.my-field1, mt.my-field2  
FROM my-table AS mt  
↳ SELECT name, country FROM competitors;
```

- Sélectionner le champ **score** de la table **competitors** et supprimer les doublons :

score
3200
399
150

(3 lignes)

```
SELECT DISTINCT score FROM competitors;
```

- Sélectionner le champ **country** de la table **competitors** (sans doublons) et trier l'affichage par ordre décroissant :

country
US
LX
FR

(3 lignes)

```
SELECT DISTINCT country FROM competitors ORDER BY  
country DESC;
```

- Sélectionner le champ **score** de la table **competitors** pour des valeurs comprises entre **200** et **1000** :

score
399

(1 ligne)

```
SELECT SCORE FROM competitors WHERE score BETWEEN 200 AND 1000;
```

- Arrondir les scores à 0.01 :

name	round
Tux	3200.00
Linus	3200.00
Bill	150.00
Boule	150.00
Steve	399.00

(5 lignes)

```
SELECT name, ROUND(score, 2) FROM competitors;
```

- Faire précéder les scores de «Résultat : » :

name	format
Tux	Résultat : 3200
Linus	Résultat : 3200
Bill	Résultat : 150
Boule	Résultat : 150
Steve	Résultat : 399

(5 lignes)

`SELECT name, FORMAT('Résultat : %s', score)
FROM Competitors`

- Remplacer NULL par 0 :

coalesce
0

(1 ligne)

`SELECT COALESCE(NULL, 0);`

2. Agrégation

- Calculer la somme des valeurs du champ `score` de la table `competitors` :

sum
7099

(1 ligne)

`SELECT SUM(score) FROM competitors;`

- Compter les occurrences des différentes valeurs du champ `score` dans la table `competitors` :

score	count
3200	2
399	1
150	2

(3 lignes)

`SELECT score, COUNT(*) FROM competitors GROUP BY score;`

- Calculer la moyenne du champ `score` pour chaque valeur du champ `country` dans la table `competitors` lorsque `country` n'est pas LX :

country	avg
US	274.5000000000000000
FR	150.0000000000000000

(2 lignes)

`SELECT country, AVG(score)
FROM competitors
GROUP BY country
HAVING country <> 'LX';`

3. Expressions de tables communes

CTE

Common Table Expression

- Stocker dans une table `my_aux_table` un résultat auxiliaire provisoire :

`WITH my_aux_table (my_id, my_value)
AS
(SELECT ...)`

`SELECT ...`

Techniques spécifiques

competitors

competitor_id	name	country	score
1	Tux	LX	3200
2	Linus	LX	3200
3	Bill	US	150
4	Boule	FR	150
5	Steve	US	399

(5 rows)

1. Conditions

- Ajouter des mentions aux résultats :

name	score	case
Tux	3200	Bon
Linus	3200	Bon
Bill	150	Mauvais
Boule	150	Mauvais
Steve	399	Mauvais

(5 rows)

```
SELECT name, score
CASE
  WHEN score > 1000 THEN 'Bon'
  WHEN score = 1000 THEN 'Moyen'
  ELSE 'Mauvais'
END
FROM competitors;
```

2. Dates

La différence entre deux dates sous la forme AAAA-MM-JJ donne un nombre entier.

On peut aussi ajouter un nombre entier à une date de cette forme, pour obtenir une autre date.



SQL s'occupe de tout, même des années bissextiles.

- Extraire l'année et le mois d'une date :

```
SELECT TO_CHAR(date, 'YYYY-MM') AS month;
```

3. Numérotation

- Faire un classement numéroté des compétiteurs sur la table ci-dessus :

name	score	ranking
Tux	3200	1
Linus	3200	2
Steve	399	3
Bill	150	4
Boule	150	5

(5 lignes)

```
SELECT name, score
ROW_NUMBER() OVER(
  ORDER BY score DESC
) AS RANKING
FROM competitors;
```

4. Sommes cumulées

- Calculer des sommes cumulées :

name	score	cumulative_sum
Bill	150	150
Boule	150	300
Linus	3200	3500
Steve	399	3899
Tux	3200	7099

(5 lignes)

```
SELECT name, score, SUM(score) OVER (
  ORDER BY name
) AS cumulative_sum
FROM competitors;
```

5. Partitions

- Rappel : **GROUP BY** produit une ligne par regroupement :

country	avg
LX	3200.0000000000000000
US	274.5000000000000000
FR	150.0000000000000000

(3 lignes)

```
SELECT country, AVG(score)
FROM competitors
GROUP BY country;
```

- Au contraire, **PARTITION BY** ne fait pas de regroupement :

name	country	avg
Boule	FR	150.0000000000000000
Tux	LX	3200.0000000000000000
Linus	LX	3200.0000000000000000
Bill	US	274.5000000000000000
Steve	US	274.5000000000000000

(5 lignes)

```
SELECT name, country,
  AVG(score) OVER (PARTITION BY
    country)
FROM competitors
```

- Ainsi, on peut obtenir des sommes cumulées par partition de la façon suivante :

name	country	sum
Boule	FR	150
Linus	LX	3200
Tux	LX	6400
Bill	US	150
Steve	US	549

(5 lignes)

```
SELECT name, country, SUM(score) OVER (
  PARTITION BY country
  ORDER BY name
)
FROM competitors;
```

Consignes pour les travaux pratiques

Début de TP

Clonage du dépôt Git

```
git clone https://git.unilim.fr/votre-dépôt-r2.06.git
cd votre-dépôt-r2.06
```

Connexion au serveur PostgreSQL

1. Connectez-vous au serveur Postgres ainsi :

psql

2. Tapez la commande `\?` pour voir les commandes disponibles.



La touche `Espace` permet de changer de page, et la touche `B` de revenir en arrière.

3. Tapez la commande `\h` pour obtenir un aide-mémoire SQL.
4. Puis tapez par exemple `\h CREATE DATABASE` pour voir la fiche de documentation de l'instruction `CREATE DATABASE`.

<i>\c my-base my-role</i>	se connecter à <code>my_base</code> en tant que <code>my_role</code>
<i>\l+</i>	lister les bases de données
<i>\du+</i>	lister les rôles
<i>\dn+</i>	lister les schémas
<i>\dt+</i>	lister les tables
<i>\dp+</i>	lister les permissions
<i>\d+ my-table</i>	examiner la structure de <code>my_table</code>

Pendant le TP



- Mettez bien le point-virgule à la fin de chaque instruction SQL.
- Utilisez la complétion automatique du terminal (touche `=`).
- Rappelez les commandes précédentes au moyen de la touche `↑`;
- Tapez `Ctrl + A` pour aller au début de la ligne de commande.
- Tapez `Ctrl + E` pour aller à la fin de la ligne de commande.

Exécution de commandes à distance

Directement

psql -c 'DROP TABLE my-table;'

Depuis un fichier SQL (y compris une sauvegarde)

psql < my-file.sql

Fin de TP

Sauvegardes diverses

- Sauvegarde de la base `my_database` :

mkdir tp1

pg_dump -U my-user -d my-database > tp1/my-database.sql

- Sauvegarde de toutes les bases et de tous les rôles :

mkdir tp1

pg_dumpall -U my-super-user > tp1/my-databases-and-roles.sql

Synchronisation Git

1. Ajoutez les sauvegardes à l'index de Git :

`git add *`

2. Vérifiez l'état de l'index de Git :

`git status`

3. Créez une validation :

`git commit -m "message explicatif"`

4. Envoyez vos validations sur la forge :

`git push`

5. Vérifiez que vos fichiers et vos validations apparaissent bien sur <https://git.unilim.fr/votre-dépôt-r2.06.git>.